NAME: RICHARD BENSON RAJ.B.A

REG NO: 1925260077

COURSE CODE: CSA1711

COURSE: ARTIFICIAL INTELLIGENCE

ASSESSMENT TOOL-1 CO1

1. Bridge and Torch Problem – State-Space Formulation and UCS

i. Problem Formulation

People: A = 1 min, B = 2 min, C = 5 min, D = 10 min.

State Representation:
A state is represented as:

(Left side people, Right side people, Torch position)

Example:
({A,B,C,D}, {}, L) → Everyone is on the left and torch is on the left.

Initial State:
({A,B,C,D}, {}, L)

Goal State:
({}, {A,B,C,D}, R)

Actions:

- Two people can cross together.
- One person can cross alone.
- The torch must be on the side from which they cross.
- Maximum 2 people can be on the bridge.

Transition Model:
When two people cross, they move from one side to the other along with the torch.

The crossing time is the **slower person's time**.

Examples:

- A + B → 2 minutes
- A + C → 5 minutes
- A + D → 10 minutes

Goal Test:

Check whether all four people are on the right side:

({}, {A,B,C,D}, R)

Path Cost:

Total time taken for all crossings.

ii. Uniform Cost Search (UCS)

UCS always selects the state having the **lowest total cost so far**.

The optimal sequence is:

Step	Action	Time	Total
1	A + B cross →	2	2
2	A returns ←	1	3
3	C + D cross →	10	13
4	B returns ←	2	15

Step	Action	Time	Total
5	A + B cross →	2	**17**

Optimal Schedule

A+B →, A ←, C+D →, B ←, A+B →

Minimum total time = 17 minutes.

So, UCS finds the **optimal cost = 17 minutes**.

iii. Why Fastest-Person Strategy is Not Optimal

The obvious strategy is to make the fastest person **A (1 minute)** escort everyone.

For example:

- A+B → = 2
- A ← = 1
- A+C → = 5
- A ← = 1
- A+D → = 10

Total = 19 minutes

This is **not optimal** because C and D can cross together in **10 minutes**, instead of making D cross separately with A.

Therefore:

Fastest-person strategy = 19 minutes
UCS optimal solution = 17 minutes

Final Answer

Minimum crossing time = 17 minutes.
Optimal sequence: A+B →, A ←, C+D →, B ←, A+B →.

3. Exam Timetabling – Constraint Satisfaction Problem (CSP)

i. Variables, Domains and Constraints

Variables:
The six exams are the variables:

AI, DBMS, OS, CN, DAA, SE

Domains:
Each exam can be scheduled in any of the three slots:

{T1, T2, T3}

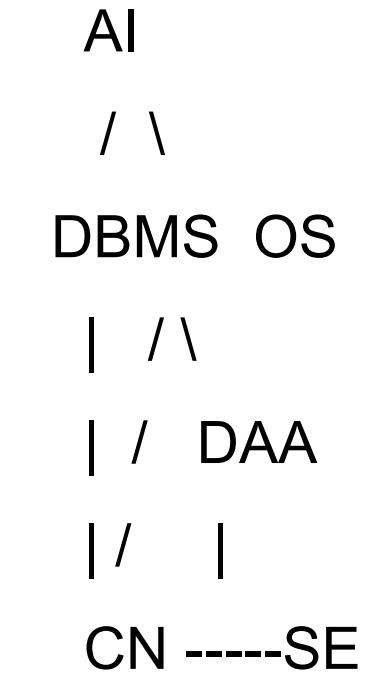
Constraints:

Conflicting exams **cannot be in the same time slot**.

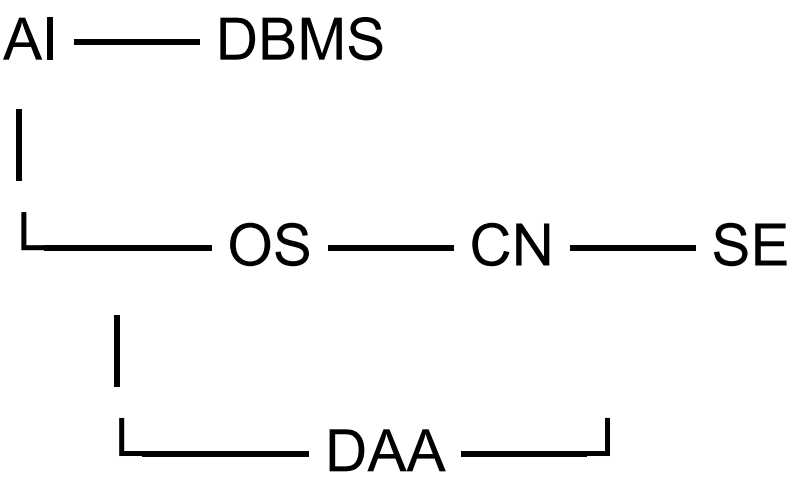
- AI ≠ DBMS
- AI ≠ OS
- DBMS ≠ CN
- OS ≠ CN
- OS ≠ DAA
- CN ≠ SE
- DAA ≠ SE

Constraint Graph

Each node represents an exam, and an edge represents a conflict.



More clearly, the edges are:



DBMS —— CN

---

ii. Backtracking Search with MRV and Forward Checking

Initially:

AI = {T1,T2,T3}

DBMS = {T1,T2,T3}

OS = {T1,T2,T3}

CN = {T1,T2,T3}

DAA = {T1,T2,T3}

SE = {T1,T2,T3}

Since all have 3 values initially, choose a highly constrained variable. Using **MRV + degree tie-breaking**, choose **OS**.

Step 1: Assign OS = T1

Forward checking removes T1 from OS's neighbors:

OS = T1

AI = {T2,T3}

CN = {T2,T3}

DAA = {T2,T3}

Choose **AI**.

Step 2: Assign AI = T2

AI = T2

DBMS = {T1,T3}

Choose **CN** because it has multiple constraints.

Step 3: Assign CN = T2

Since CN conflicts with DBMS, OS and SE:

CN = T2

DBMS = {T1,T3}

SE = {T1,T3}

Choose **DAA**.

Step 4: Assign DAA = T2

DAA conflicts with OS and SE:

DAA = T2

SE = {T1,T3}

Choose **DBMS**.

Step 5: Assign DBMS = T1

DBMS = T1

Choose **SE**.

Step 6: Assign SE = T1

Check all constraints:

- AI(T2) ≠ DBMS(T1) ✓
- AI(T2) ≠ OS(T1) ✓
- DBMS(T1) ≠ CN(T2) ✓
- OS(T1) ≠ CN(T2) ✓
- OS(T1) ≠ DAA(T2) ✓
- CN(T2) ≠ SE(T1) ✓
- DAA(T2) ≠ SE(T1) ✓

Final Timetable

Exam Slot

AI	T2
DBMS	T1
OS	T1
CN	T2
DAA	T2

Exam Slot

SE T1

Solution:

T1 → DBMS, OS, SE

T2 → AI, CN, DAA

T3 → Empty

All constraints are satisfied.

---

iii. Arc Consistency (AC-3)

AC-3 removes values from a variable's domain when they cannot satisfy a constraint with a neighboring variable.

For example:

OS ≠ CN

If:

OS = T1

then AC-3 removes T1 from CN's domain:

CN = {T2,T3}

Similarly, if:

CN = T2

then AC-3 removes T2 from:

DBMS and SE

This process continues through the constraint graph.

Advantages of AC-3

1. Removes inconsistent values early.
2. Reduces variable domains.
3. Detects conflicts before deep backtracking.
4. Reduces the number of nodes explored.
5. Makes backtracking search faster.

Conclusion

MRV chooses the variable with the fewest remaining values, **forward checking** removes conflicting values after each assignment, and **AC-3** performs stronger domain filtering before/during search. Together, they significantly reduce the search effort.

4. Route Finding with A* Search

Given

Edges and costs:

- S → A = 2
- S → B = 5
- A → C = 4
- A → D = 7
- B → D = 3
- C → D = 1
- C → G = 6
- D → G = 2

Heuristic values:

Node h(n)

S 7

A 6

B 4

Node h(n)

C 3

D 2

G 0

A* uses:

f(n) = g(n) + h(n)

where g(n) = cost from S to n.

---

i. A* Search

Step 1: Start with S

For S:

g(S) = 0

f(S) = 0 + 7 = 7

Open: [S(7)]
Closed: []

Expand S:

- A: g = 2, h = 6 → f = 8
- B: g = 5, h = 4 → f = 9

Open: [A(8), B(9)]
Closed: [S]

Step 2: Expand A

A has the lowest f-value.

For C:

g(C) = 2 + 4 = 6

f(C) = 6 + 3 = 9

For D:

g(D) = 2 + 7 = 9

f(D) = 9 + 2 = 11

Open: [B(9), C(9), D(11)]
Closed: [S, A]

Step 3: Expand B

B has f = 9.

B → D:

g(D) = 5 + 3 = 8

This is better than previous g(D)=9.

Therefore update D:

f(D) = 8 + 2 = 10

Open: [C(9), D(10)]
Closed: [S, A, B]

Step 4: Expand C

C has f = 9.

C → D:

g(D) = 6 + 1 = 7

This is better than 8.

So update D:

f(D) = 7 + 2 = 9

C → G:

g(G) = 6 + 6 = 12

f(G) = 12 + 0 = 12

Open: [D(9), G(12)]
Closed: [S, A, B, C]

Step 5: Expand D

D has f = 9.

D → G:

g(G) = 7 + 2 = 9

f(G) = 9 + 0 = 9

Update G.

Open: [G(9)]
Closed: [S, A, B, C, D]

Step 6: Expand G

G is the goal.

Open: []
Closed: [S, A, B, C, D, G]

A* Search Summary

Step	Expanded	Open List	Closed List
1	S	A(8), B(9)	S

Step	Expanded	Open List	Closed List
2	A	B(9), C(9), D(11)	S,A
3	B	C(9), D(10)	S,A,B
4	C	D(9), G(12)	S,A,B,C
5	D	G(9)	S,A,B,C,D
6	G	—	S,A,B,C,D, G

---

ii. Path and Cost

The best path is:

S → A → C → D → G

Cost:

S → A = 2

A → C = 4

C → D = 1

D → G = 2

Total = 2 + 4 + 1 + 2 = 9

Answer:

Shortest path = S → A → C → D → G

Minimum cost = 9

---

iii. Admissibility and Consistency

Admissible Heuristic

A heuristic is **admissible** if:

h(n) ≤ actual minimum cost from n to G

Actual costs:

Node	Actual cost to G	h(n)	Admissible?
S	9	7	✓
A	7	6	✓
B	5	4	✓
C	3	3	✓
D	2	2	✓
G	0	0	✓

Therefore:

The heuristic is admissible. ✓

Consistent Heuristic

For every edge:

$h(n) \leq cost(n,m) + h(m)$

Check important edges:

- S → A: $7 \leq 2 + 6 = 8$ ✓
- S → B: $7 \leq 5 + 4 = 9$ ✓
- A → C: $6 \leq 4 + 3 = 7$ ✓
- A → D: $6 \leq 7 + 2 = 9$ ✓
- B → D: $4 \leq 3 + 2 = 5$ ✓
- C → D: $3 \leq 1 + 2 = 3$ ✓
- C → G: $3 \leq 6 + 0 = 6$ ✓
- D → G: $2 \leq 2 + 0 = 2$ ✓

Therefore:

The heuristic is consistent. ✓

---

iv. Greedy Best-First Search

Greedy Best-First Search uses only:

$f(n) = h(n)$

It ignores the path cost g(n).

Step 1

From S:

- A → h = 6
- B → h = 4

Choose **B** because h(B) = 4.

Step 2

From B:

- D → h = 2

Choose **D**.

Step 3

From D:

- G → h = 0

Choose **G**.

Therefore, Greedy finds:

S → B → D → G

Cost:

5 + 3 + 2 = 10

A* vs Greedy Best-First Search

Feature	A*	Greedy Best-First
Formula	f = g + h	f = h
Path found	S → A → C → D → G	S → B → D → G
Cost	**9**	**10**
Optimal?	**Yes**	Not necessarily
Uses path cost?	Yes	No

Final Conclusion

A* finds the optimal path with cost **9**:

S → A → C → D → G

Greedy Best-First Search finds a path with cost **10**:

S → B → D → G

So, **A* gives the better/optimal route because it considers both actual cost and heuristic cost.**

10. Problem Formulation: Tower of Hanoi

There are **3 pegs: A, B, C** and **3 discs**. Initially, all discs are on **A**, with the largest at the bottom.

Let:

- **D1** = smallest disc
- **D2** = medium disc
- **D3** = largest disc

---

i. Problem Formulation

1. States

A state represents the position of all three discs on the three pegs.

Example:

(ABC) means:

- D1 on A
- D2 on B
- D3 on C

Initial state:

(AAA)

Goal state:

(CCC)

2. Initial State

All discs are on peg A:

A	B	C
D1	-	-
D2		
D3		

Initial state = (AAA)

3. Actions

A disc can be moved:

- From A → B
- From A → C

- From B → A
- From B → C
- From C → A
- From C → B

Rules:

1. Only one disc can be moved at a time.
2. Only the top disc can be moved.
3. A larger disc cannot be placed on a smaller disc.

4. Transition Model

Moving one legal top disc from one peg to another produces a new state.

Example:

AAA → BAA

This means D1 is moved from A to B.

5. Goal Test

Check whether all discs are on C:

CCC

Therefore:

Goal = (CCC)

6. Path Cost

Each legal disc movement has a cost of **1**.

Therefore:

Path cost = Number of moves

---

ii. State Space – First Three Levels

Starting state:

Level 0:

A

———

D1

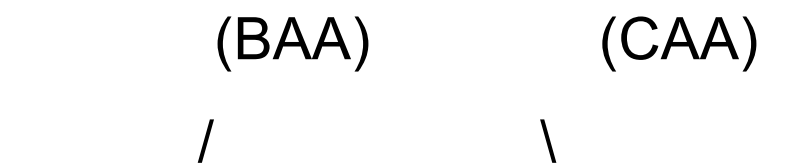
D2

D3

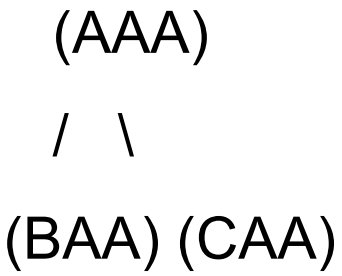
(AAA)

Level 1

Move the smallest disc D1 from A:



So:



Level 2

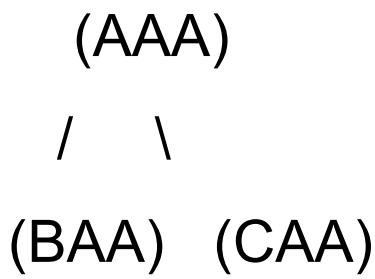
From (BAA), D2 can move from A to C:

(BAA) → (BCA)

From (CAA), D2 can move from A to B:

(CAA) → (CBA)

Thus the useful state-space tree is:



| |

(BCA) (CBA)

Note: Some states may also have moves that return to a previous state. In a search tree, repeated states can be avoided.

---

iii. Minimum Sequence of Moves

For **3 discs**, the minimum number of moves is:

$2^3 - 1 = 7$ moves

Steps

Move	Action
1	D1: A → C
2	D2: A → B
3	D1: C → B
4	D3: A → C

Move	Action
5	D1: B → A
6	D2: B → C
7	D1: A → C

Sequence

A → C

A → B

C → B

A → C

B → A

B → C

A → C

General Formula

For **n discs**:

$T(n) = 2T(n-1) + 1$

Solving this recurrence:

$T(n) = 2^n - 1$

Examples:

Number of discs	Minimum moves
1	1
2	3
3	7
4	15
n	**$2^n - 1$**

Therefore, for 3 discs:

$2^3 - 1 = 7$ moves.

---

iv. DFS or BFS?

BFS (Breadth-First Search)

BFS explores states level by level.

Since every move has the same cost (**1**), BFS can find the solution with the **minimum number of moves**.

DFS (Depth-First Search)

DFS explores one path deeply before trying another path.

It may find a solution, but **it does not guarantee the shortest solution** unless additional conditions are used.

Which is suitable?

BFS is more suitable when the requirement is to guarantee the minimum number of moves, because all actions have equal cost.

However, Tower of Hanoi has a very regular recursive structure, so **DFS/recursive solution is much more memory-efficient and natural for solving it directly**.

Final Answer

For state-space search → BFS is preferable for guaranteed minimum moves.

For an actual Tower of Hanoi program → DFS/recursion is more efficient and commonly used.

14. OLA Cab Booking – Problem-Solving Agent

Type of Problem-Solving Agent

The suitable agent is a **Goal-Based Agent**.

A goal-based agent selects actions to achieve a specific goal. Here, the customer's goal is to **travel from the current location to the destination** using OLA.

It can consider:

- Current location
- Destination
- Cab availability
- Cab type
- Distance
- Estimated time

- Cost
- Customer's preference

Problem Formulation

Initial State: Customer is at the pickup location.

Goal State: Customer reaches the destination.

Actions: Select cab, book cab, travel through route, and reach destination.

Path Cost: Fare, distance, or travel time.

Pseudocode

START

Input current_location

Input destination

Display available cab types

1. Mini

2. Micro
3. Sedan
4. Shared
5. Prime

Input cab_type

Find available cabs near current_location

IF cab is available THEN
Select the cab
Calculate fare and estimated time
Display fare and time

IF customer confirms booking THEN
Book the cab
Assign driver
Start journey

```
        Find route to destination

        WHILE destination not reached
            Follow the route
        END WHILE

        Display "You have reached the
destination"
    ELSE
        Display "Booking cancelled"
    END IF

ELSE
    Display "No cabs available"
END IF

STOP
```

Simple Explanation

Customer enters pickup & destination

↓

Select cab type

↓

Check cab availability

↓

Cab available?

/ \

Yes No

↓ ↓

Show fare No cab found

↓

Confirm booking

↓

Assign driver

↓

Find route

↓

Reach destination

Final Answer

Problem-Solving Agent: Goal-Based Agent

Goal: Reach the customer's destination using a suitable OLA cab.

The agent chooses suitable actions based on the customer's destination, cab type, availability, cost, and route to achieve the goal.